

Lecture 3.1 – 3.2

Shortest path

Application

Dijkstra's algorithm

- Dijkstra's algorithm is a graph search algorithm that solves single-source shortest path for a graph with nonnegative weights.
- Widely used in network routing protocol, e.g., Open Shortest Path First (OSPF) protocol.

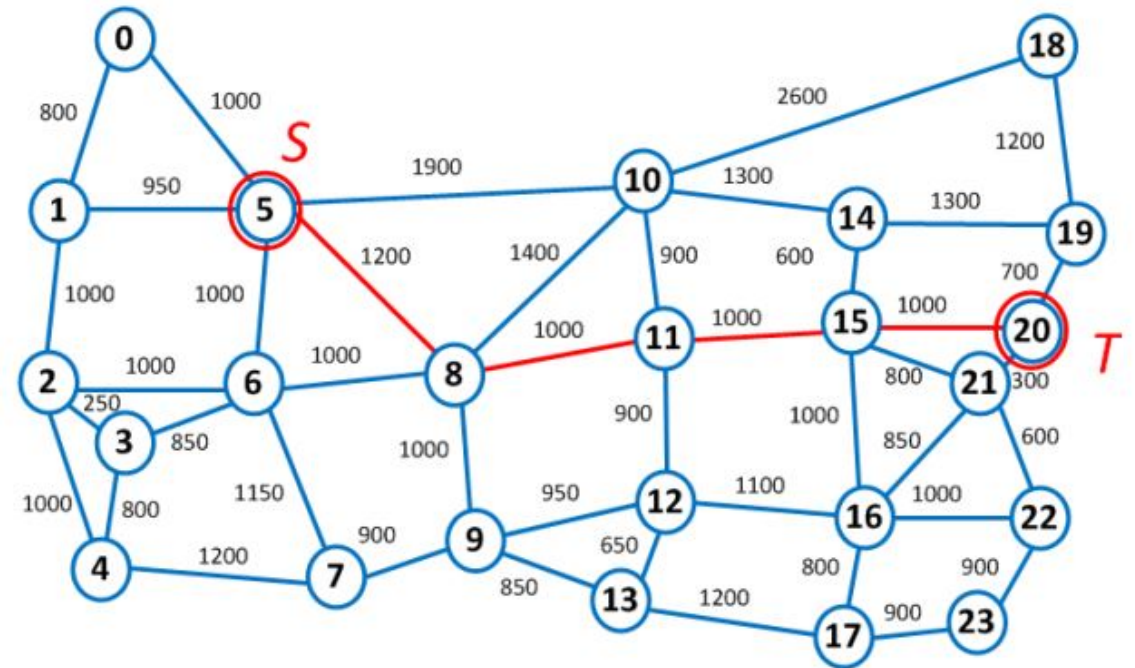


Fig. 1 24-node U.S. mesh network

Algorithm

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```

RELAX(u, v, w)

```
1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
```

Algorithm

```
function Dijkstra(Graph, source):  
    for each vertex v in Graph:  
        distance[v] = infinity  
  
    distance[source] = 0  
    G = the set of all nodes of the Graph  
  
    while G is non-empty:  
        Q = node in G with the least dist[ ]  
        mark Q visited  
        for each neighbor N of Q:  
            alt_dist = distance[Q] + dist_between(Q, N)  
            if alt_dist < distance[N]  
                distance[N] := alt_dist  
  
    return distance[ ]
```

Complexity of Priority Queue

Priority Queue Data Structure	Push Node	Pop Node	Decrease Key
Array	$O(N)$	$O(N)$	$O(N)$
Binary Heap	$O(\log(N))$	$O(\log(N))$	$O(\log(N))$
Binomial Heap	$O(1)$	$O(\log(N))$	$O(\log(N))$
Fibonacci Heap	$O(1)$	$O(\log(N))$	$O(1)$

Complexity?

With decrease-key operation:

- V push operation
- V remove operation
- E decrease-key operation

Without decrease-key operation:

- E insert operation
- E remove operation

Complexity?

With decrease-key:

- Array: $O(V^2 + EV)$
- **Binary heap: $O(V \log V + E \log V) = O((E+V) \log V)$**
- Fibonacci heap: $O(V \log V + E)$

Without decrease-key:

- Binary heap: $O(E \log V + E \log V) = O(E \log V)$

Goal of the Proof

- Assume, S : set of nodes whose shortest distances are finalized. We need to show that when a node v is added to S :

$$d[v] = \delta(s, v)$$

- $d[v]$: Shortest distance to node v , calculated by the algorithm
- $\delta(s, v)$ is the true shortest distance from s to v

Trivially we can say, the calculated distance can't be better than actual shortest distance:

$$d[v] \geq \delta(s, v) \dots \text{(eq: 1)}$$

Proof Outline (by Induction)

- Base case: $d[s] = \delta(s, s) = 0$ (trivial)
- Inductive step:
 - Assume $d[u] = \delta(s, u)$ for all u in S
 - Show that for the next node v selected: $d[v] = \delta(s, v)$

Proof Continue...

Let P be a shortest path from s to v , and let u be the last node on P such that $u \in S$ and the next node $w \notin S$.

$$s \rightsquigarrow u \rightarrow w \rightarrow \cdots \rightarrow v$$

Since $u \in S$, by the inductive hypothesis:

$$d[u] = \delta(s, u)$$

Proof Continue...

- After relaxing (u, w) , where $u \in S$ and (u, w) is on shortest path:
$$d[w] \leq d[u] + w(u, w)$$
- Since $d[u] = \delta(s, u)$, we get:
$$d[w] \leq \delta(s, u) + w(u, w)$$
- Since edge (u, w) is in the shortest path:
$$\delta(s, w) = \delta(s, u) + w(u, w)$$
- So:
$$d[w] \leq \delta(s, w) \text{ (eq: 2)}$$

Proof Continue...

- As the algorithm picks v , before w , so:
$$d[v] \leq d[w] \dots (\text{eq: 3})$$
- Combining equation 2, and equation 3:
$$d[v] \leq \delta(s, w) \dots (\text{eq: 4})$$
- Now: $\delta(s, v) = \delta(s, w) + \text{cost}(w \rightarrow v)$, and every edge is non-negative. So the cost will be greater or equal to 0. So:
$$\delta(s, v) \geq \delta(s, w)$$
- From equation 4:
$$d[v] \leq \delta(s, v) \dots (\text{eq: 5})$$

Proof Continue...

We have proved both of these equations:

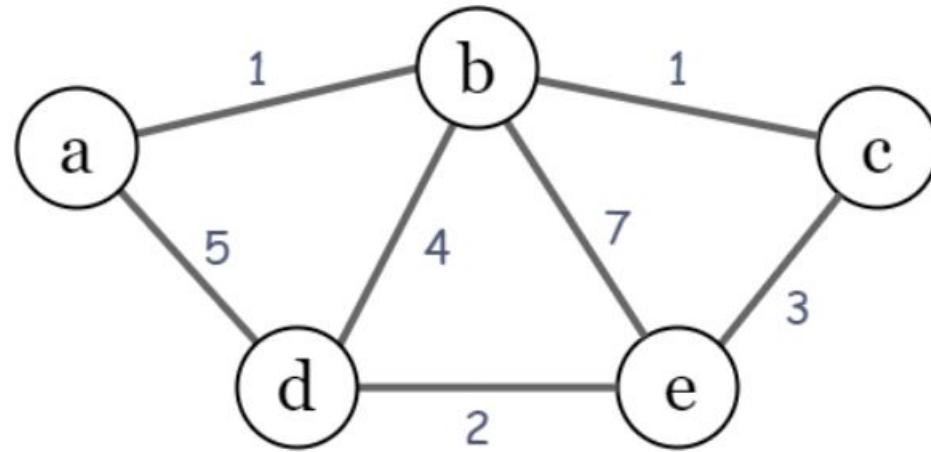
$$d[v] \geq \delta(s, v) \dots (\text{eq: 1})$$

$$d[v] \leq \delta(s, v) \dots (\text{eq: 5})$$

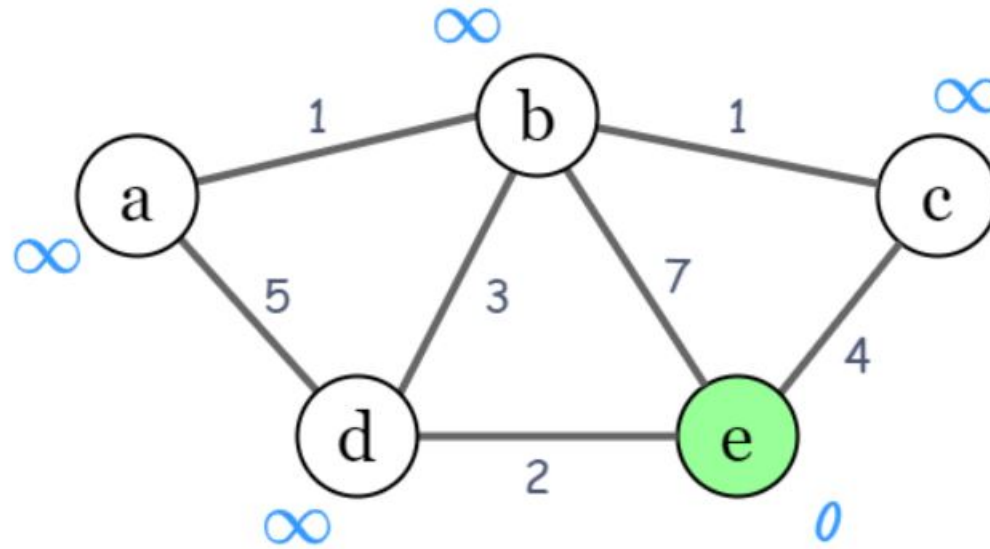
Both equation can only be true, if:

$$\mathbf{d[v] = \delta(s, v)}$$

Example



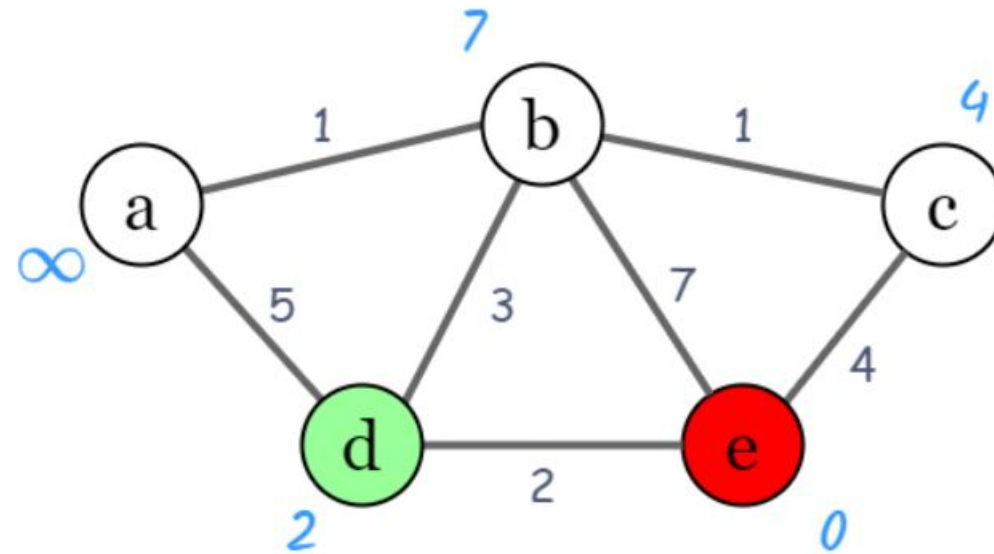
Example



$$Q = \{(e, 0), (a, \infty), (b, \infty), (c, \infty), (d, \infty)\}$$

$$S = \emptyset$$

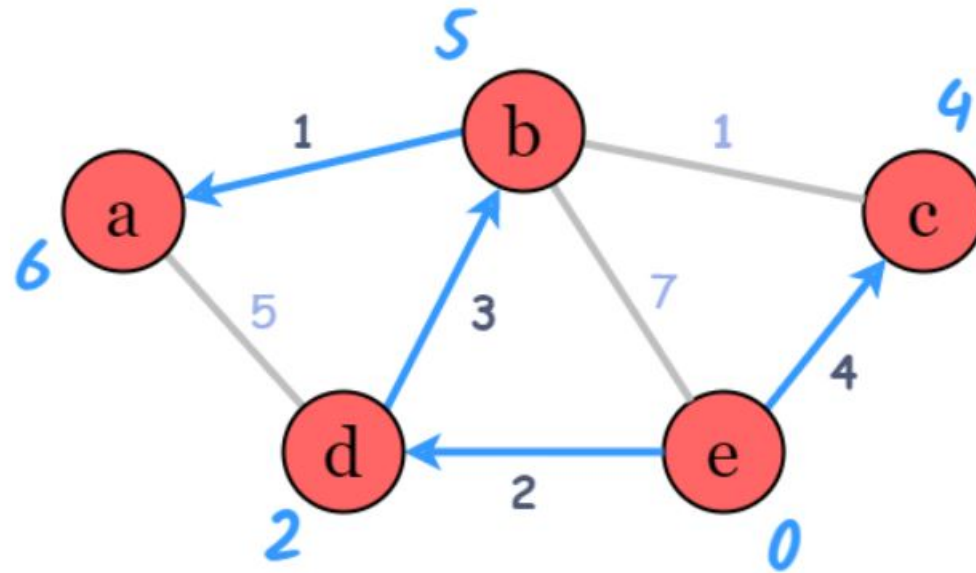
Example



$$Q = \{(d, 2), (c, 4), (b, 7), (a, \infty)\}$$

$$S = \{e\}$$

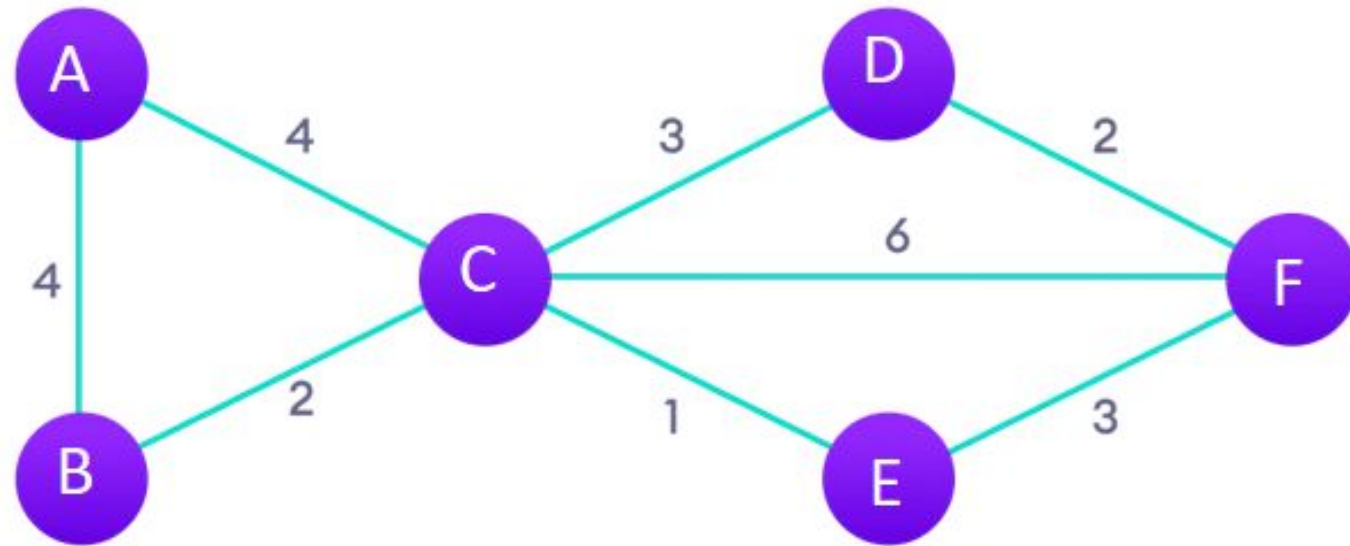
Example



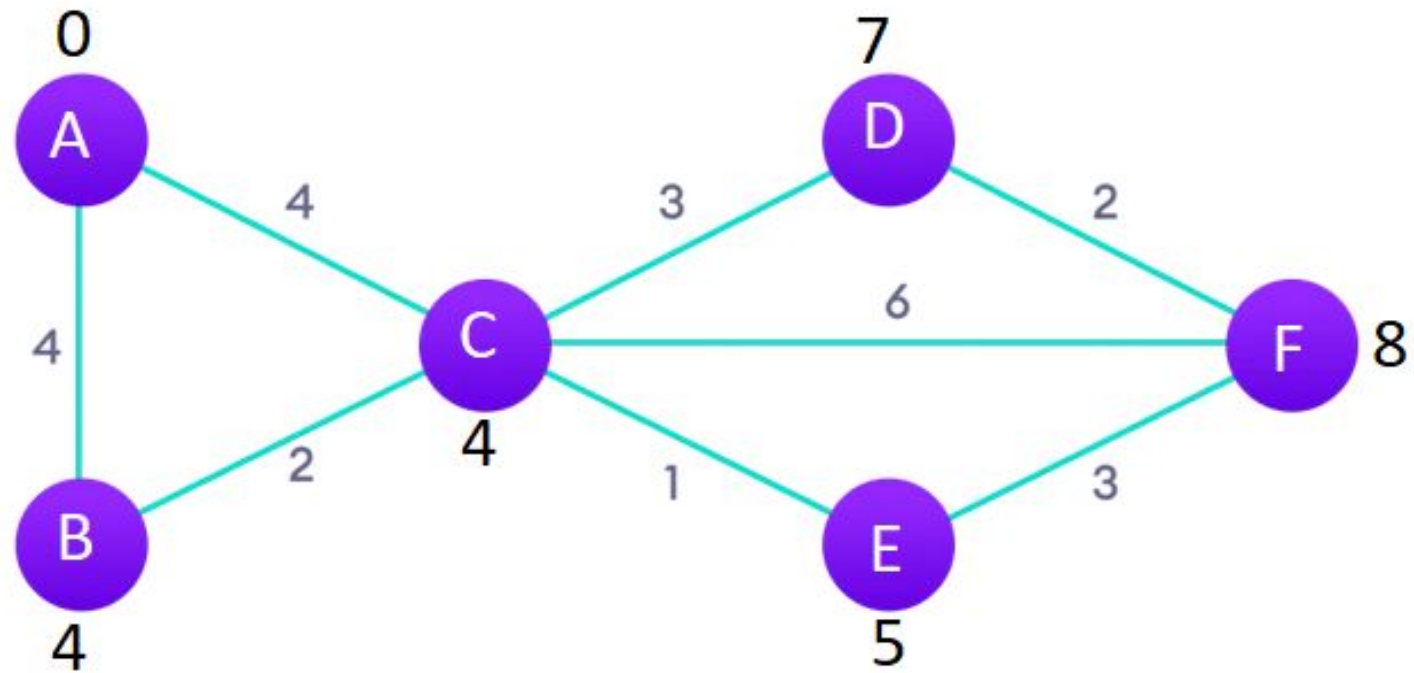
$$Q = \{\}$$

$$S = \{e, d, c, b, a\}$$

Example



Example



Implementation

- **typedef** pair<long long, long long> **pr**;
- vector<**pr**> adj[MAX]; // For adjacency list
- void addEdge(int u, int v, int w){
 adj[u].push_back(make_pair(**w**,v));
 adj[v].push_back(make_pair(**w**,u));
}

Implementation

- `priority_queue< pr > Q; // default sorting is descending`
- `priority_queue<pr, vector<pr>, greater<pr> > Q;`

Implementation

```
for(int i = 1; i <= n; i++){  
    dist[i] = infinity;  
}  
dist[src] = 0;  
Q.push(make_pair(dist[src], src));
```

Few practice problems

- CSES 1671
 - <https://cses.fi/problemset/task/1671>
- Codeforces 449B
 - <https://codeforces.com/problemset/problem/449/B>
- Codeforces 20C
 - <https://codeforces.com/problemset/problem/20/C/>
- UVA 10986
 - https://onlinejudge.org/index.php?option=onlinejudge&Itemid=8&page=show_problem&problem=1927